FOOM归因方案

背景

因线上 FOOM 的数据量太大（目前统计大概在日均700～1000次左右）。所以对目前我们项目统计 FOOM 的方案进行了分析。

同时，结合对微信 matrix 库的调研，发现对 FOOM 的细节处理还是有一小部分差异。

所以进行了简单整理，仅供参考：

现状

当前Gravity是如何判定上一次是否出现FOOM?

1. **App当时在前台运行**
2. **App没有出现crash**
3. **App没有升级**
4. **系统没有升级**

```
// 1. foreground
// 2. not crash
// 3. not app upgrade
// 4. not system upgrade
return lastRunAppAnalyticsData.isAppForeground &&
!isLastRunCrashed &&
lastRunAppAnalyticsData.appVersion == currentAppVer &&
lastRunAppAnalyticsData.osVersion == currentOSVer
```

外部方案：Facebook

Facebook是如何判定上一次是否出现FOOM?

1. **App没有升级**
2. **App没有调用exit()或abort()退出**
3. **App没有出现crash**
4. **用户没有强退App**
5. **系统没有升级/重启**
6. **App当时没有后台运行**

7. **App出现FOOM**

存在的细节case：

1. 对比上述方案的差异。所以对调用了**exit(0)**函数和使用gravity时发生了**关闭重启**。下一次使用时，也会当作一次FOOM上报上去。
2. 除此差异之外，其他相同的条目，3crash统计依赖于crash捕获组件，6、7依赖于ApplicationState和前后台切换通知。这些东西也会因一些特殊case，而导致数据的差异化。
   a. **ApplicationState不准**

      部分系统会在后台短暂唤起app，ApplicationState是Active，但又不是BackgroundFetch；执行完didFinishLaunchingWithOptions就退出了，也有收到BecomeActive通知，但很快也退出；整个启动过程持续5～8秒不等。

      AI给出的导致在后台被短暂唤醒的case：

      在iOS中，确实存在某些情况下系统会在后台短暂地唤醒应用程序，这通常是为了执行一些后台处理的任务。一些可能的情况包括：
      1. **位置更新**: 如果应用注册了地理位置更新，并且设置了相应的后台模式，系统可能会在适当的位置变化后唤醒应用执行代码。
      2. **推送通知**: 当收到静默推送通知（Silent Push Notification）时，如果应用设置了后台模式，系统会唤醒应用来处理推送通知，即使用户没有直接与通知互动。
      3. **Background App Refresh**: 这是一种后台模式，允许应用程序在后台定期更新内容。但通常这种唤醒不会使应用状态变为`.active`。
      4. **外围设备**: 对于使用蓝牙和其他外围设备的应用，当外围设备发出信号时，系统可能会唤醒应用来处理这些信号。

      在上述情况下，应用程序的`applicationState`在被唤醒期间可能会是`.active`，尽管用户实际上并没有直接与应用程序进行交互。这是因为从系统的角度看，应用程序已经被激活以执行某些操作。
      `didFinishLaunchingWithOptions`方法在应用程序启动时调用，不管是用户直接点击图标启动，还是通过通知或其他事件被系统唤醒。而`applicationDidBecomeActive:`是应用程序变成活跃状态时调用，这在后台任务执行完毕即将终止之前可能会触发。
      这种后台唤醒与前台唤醒是有区别的，并不意味着用户开始与应用程序交互。若要正确响应这些唤醒，你需要在应用程序的delegate方法中适当处理后台任务，并注意使用正确的API来判断你的应用程序是被后台事件唤醒还是用户直接唤醒的。

      解决方法是收到BecomeActive通知一秒后，才认为这次启动是正常的前台启动。这方法只能减少误判概率，并不能彻底解决。

   b. **crash组件是否有统计全crash?**

      这个点，目前我们是使用firebase库对崩溃进行统计的，所以在不确定它内部实现的情况下，不太好下结论。

      这里列出来，只是确实存在这种可能性。 微信举了一个例子：比如，在收到crash信号，crash库写日志时，内存池出现了问题，组件无法正常写入，甚至出现了二次crash。

      这里给出解决方案是：不依赖crash回调，只要本地存在上一次的crashlog（不管是否完整），就认为是crash引起的app重启。

   c. **前台卡死引起系统watchdog强杀**

      如果前台由于线程过多、死锁、CPU使用率过高等，会导致被系统watchdog强杀。这类强杀也是无法被App捕获的。

      这类case可能是比较多的，后期需要关注的。

      解决方案：结合卡顿捕获来处理，如果在前台运行的最后一刻捕获到卡顿，就认为是“APP前台卡顿导致重启”。

结论

在当前我们的DAU不算多，FOOM数据量太大的情况下，前期可能无须太关心精细化的case，所以当前使用的方案，可以暂时满足当下的需求。后续对大多数内存使用优化完之后，可以考虑用Matrix的方案，尽可能将FOOM的数据校准。